

SIMULATION RESEAU

Algorithme A*

Explication du fonctionnement de l'algorithme A*
appliquer au routage dans un réseau.

 AlgoAStar.py

 Reseau

 Routage

Plan de **Présentation**

01

Structure des données

Classe de routeur et coût des liens

02

L'algorithme A*

Fonction d'évaluation $f(n) = g(n) + h(n)$

03

Processus de simulation

Génération, exécution, visualisation

04

Avantages de la méthodologie

Pourquoi l'algorithme A* est-il idéal pour le routage?

Structure des **donnees**

Les composants fondamentaux du reseau



Classe Router

Représente un nœud du réseau. Chaque routeur possède **une position (x, y)**, **un nom** (A, B, C,...,Z) et une **liste de voisins** qui lui sont connectés.



Coût des liaisons

Le coût ne dépend pas uniquement de la distance, mais il est **inversement proportionnel à la bande passante** : **1000 / débit**. -> Plus le débit est élevé => plus le coût est bas.



Cet l'algorithme permet de choisir les liens les plus rapides (**coût minimal**), afin de garantir la qualité des liens plutôt que la convergence geometrique.

L'algorithme A*

L'algorithme A* recherche le chemin optimal en utilisant une fonction d'évaluation qui combine le coût réel et l'estimation.

$$f(n) = g(n) + h(n)$$

$f(n)$

Coût total estimé

$g(n)$

Coût réel accumulé

$h(n)$

Heuristique

L'algorithme explore en priorité les noeuds où la somme $f(n)$ est la plus faible, garantissant un chemin optimal si l'heuristique est admissible ($h(n) \leq \text{coût réel}$).

Décomposition de $g(n)$ et $h(n)$



$g(n)$ — Cout reel

Coût **réellement accumulé** depuis le point de depart jusqu'au noeud actuel n .

Il prend en compte le coût total de chaque liaison parcourue : **1000 / debit** par liaison.



$h(n)$ — Heuristique

Une **estimation du coût restant** pour atteindre la destination. Le code utilise la **distance entre le routeur actuel et la destination, puis la diviser par 1000.**

Cette heuristique guide la recherche vers la destination, reduisant le nombre de noeuds explores.



L'heuristique est **admissible** ($h(n) \leq \text{coût réel}$), ce qui garantit que A* trouve toujours le chemin optimal.

Processus de **simulation**

1 Génération — `generate_network()`

Place les routeurs de manière aléatoire. Assure la **connectivité** (chaque nœud est connecté à au moins un autre). Ajoute des **liens redondants**.

2 Exécution — `run_astar()`

Utilise une **file de priorité** (`heapq`) pour une recherche efficace. Met à jour les chemins les **moins coûteux**.

3 Visualisation — `draw_network()`

Affiche le réseau avec ses nœuds et ses liens, en mettant en évidence le chemin optimal.

● Départ (Noeud A) ● Destination ● Chemin optimal A*

SECTION 4

Avantages de la méthodologie

Pourquoi A* est idéal pour le routage réseau

Dijkstra

Explore **tous les noeuds** dans toutes les directions. Précis, mais lent sur les grands réseaux. Ne fournit pas l'efficacité de la recherche heuristique $h(n)$ sur la destination.

Glouton (Greedy)

Elle ne prend en compte que des heuristiques. **Très rapide**, mais ne garantit pas un chemin optimal. Peut emprunter des détours coûteux.

A*

Combine **la précision de Dijkstra** (coût réel) avec **l'efficacité de la recherche heuristique $h(n)$** , cette méthode est à la fois optimale et rapide.



A* est l'une des méthodes les plus populaires et efficaces pour la recherche de chemin.



Merci de votre **attention**

L'algorithme A* : quand l'efficacité rencontre
l'optimalité

– Directed by Rayen –

`</>` [AlgoAStar.py](#)

 [Algorithme A*](#)

 [Documentation](#)